Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 12

Lecture – 62

Linking theory to practice

Hello and welcome to the NPTEL MOOC's course on Foundations for Virtual and Augmented Reality Systems. In this part of the lecture, we will continue our broader discussion on how to link the theoretical knowledge to the implementation environment. If you can recollect, we are discussing that with respect to an environment that is the Unity development environment. So, in that environment, What do we do?

We first install an editor interface using which we can create the system, we can deploy the system, and when we install the Unity Hub package, then automatically the Unity game engine, that is the middleware, is also installed. So, some basic concepts related to Unity and the Unity editor we have discussed in the previous parts. Also, we have seen what kind of implementation support the programmer can expect from this development environment. Today in this lecture, we are going to talk about another important component of such a development environment, that is the scripting support.

So, we are going to talk about the scripting support or other crucial programming support that the Unity Development Environment provides to a programmer for a more customized experience. So, whatever we have discussed so far is basically built-in support. If we use those, we can get a good quality experience, but that experience will not be what we want, or we might like to change it. It is whatever is provided by the system.

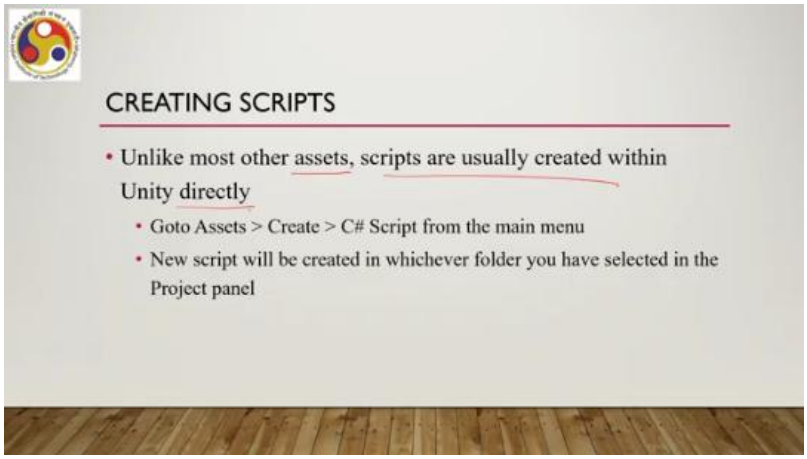
Now, if we want to have a more customized experience as per our expectations and needs, then we need some way to change the built-in support, and that way is by writing scripts or small pieces of code. So, in order to do that, first we need to set up the scripting environment, which is automatically done when you install the Unity package. So, Unity supports IDEs such as Visual Studio, Visual Studio Code, and JetBrains Rider, among others.



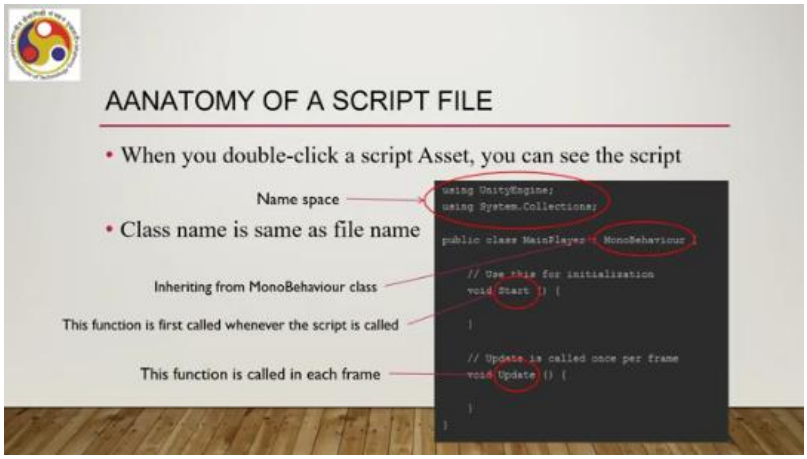
So, when you install Unity on Windows and Mac, by default Unity also installs the Visual Studio package, which is an IDE you can use to develop and compile your scripts. The scripting language supported by Unity is called C#. It is an object-oriented programming language created by Microsoft. Many of you may be familiar with this. And we can attach a C# script as a component to a game object to control the behavior or activities of the object.

Now, this addition of a script as a component allows a developer to trigger game events, modify component properties over time, and respond to user input in any way the developer wants. When you are creating a script, you are essentially creating your own new type of component that can be attached to game objects, just like any other component. So, essentially it allows you to have a component attached to a game object that will behave as per your wish rather than as per the built-in support provided by the environment.

How do we create scripts? So, unlike most other assets, scripts are usually created within Unity directly, based on using the interface provided by the IDE. So, we can go to Assets, then Create, then C# Script from the main menu to create a script. A new script will be created in whichever folder you have selected in the project panel. So, when you start a project, you select a folder and the script will be placed in that folder itself.



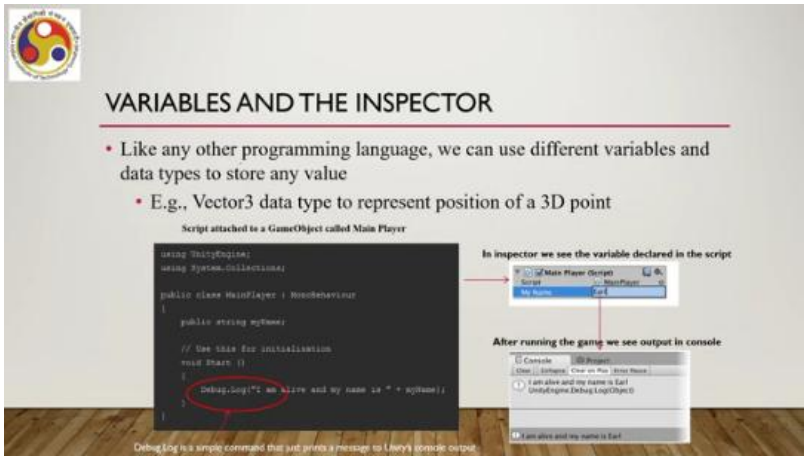
Once the script is attached to the game object as per your design or development, the script will start working whenever you start executing the game. So, how does a script file look? Let us have a look at the anatomy of a scripting file. Now, when you double-click on a script asset, you will be able to see the code within the script file. It has a top component that is the namespace, where you can see this type of syntax is used: using Unity engine, using system.collections.



So, these are some of the options available for a writer of a script. Now, the class name is the same as the file name. So, recollect that a script is an OOP language, an Object Oriented Programming language. So, it relies on the concept of a class. We are not going to discuss what a class is.

Assuming that you already have some idea of a class. Now, for this public class, you have to use this construct, and then you have to give the class name, which is the same as the file name, just like any other object-oriented programming approach. Then there is inheritance that you can do; typically, that is done. This MonoBehaviour class is basically inherited by the class you are defining. Then there is the Start function. This function is called whenever the script is called.

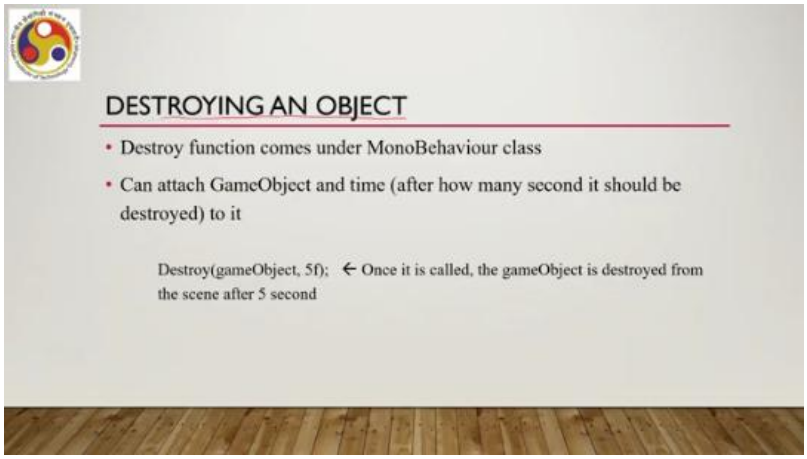
So, this is basically the starting point of the script. And then we have this Update option, which is called in each frame. So, whenever the script executes and some change happens, then this Update option is invoked; the Update function is invoked. Like any other programming language, we can use different variables and data types to store any value because scripting is nothing but a programming language.



So, variables and data types are crucial components of any programming language, and so are those for the scripting language, C#. See, for example, there is a data type Vector3, which we can use to represent a position of a 3D point; that means three coordinates can be captured in the form of this Vector3, meaning there will be three elements in the vector. Now, whatever variables you are using, say for example, 'my name' is a variable with value 'r'. So, this you can see in the Inspector interface on the Unity Hub editor. So, you can check the variable values.

Then, after running the game or the application, we can see the output on the console. So, for example, if you use this type of construct, the Debug.Log function, which just prints a message on the console output, then you can see that after the script has been executed, say, 'Here I am alive and my name is R,' something like that you can get to see. So, this development environment allows you to see the script by double-clicking on the script asset, to check the variable values separately using the Inspector interface, and also get to see the output on the console.

So, all these facilities are provided, just like any other programming environment. There is an important concept called destroying an object, supported by the scripting environment. So, there is a Destroy function that comes under the MonoBehaviour class. We can attach this function to the game object and specify a time after which it should be destroyed, typically that is in seconds.

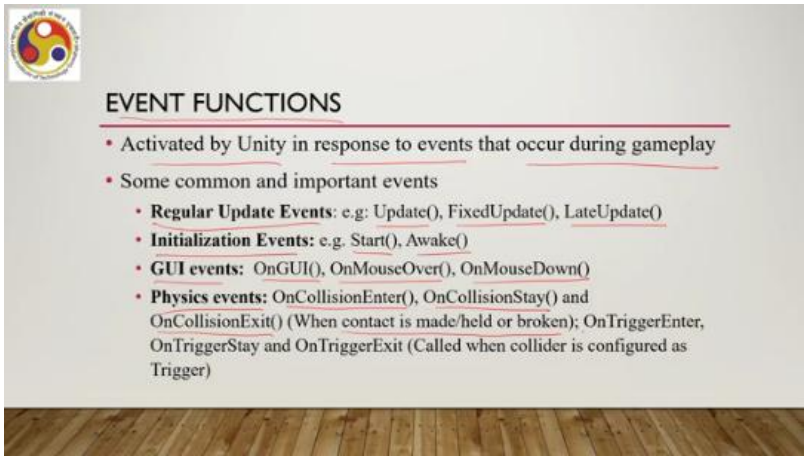


For example, if we have a function like Destroy(gameObject, 5f), that means once the function is called, the game object is destroyed from the scene after 5 seconds. So, after the function is called, after 5 seconds of that calling, the game object is destroyed, no longer visible on the scene. So, these are examples of functions. We know what a function is, typically, in a programming environment. So, once we define a function, we can use it whenever it is needed.

The function will probably require some inputs or parameters in most cases. Sometimes they may not require any parameters, which are given to the function each time it is called, just like in any other programming environment. The Start function is an example. We have already seen it where the script starts running. However, we can also define our own functions.

So, there are some prebuilt functions which are critical, which we need to use to make our script work. But there are also user-defined functions, which are typically part of any good script. There is a class of functions called event functions. These are activated by Unity in response to events that occur during gameplay. Some such common and important events are regular update events, such as the Update function, FixedUpdate function, and LateUpdate function.

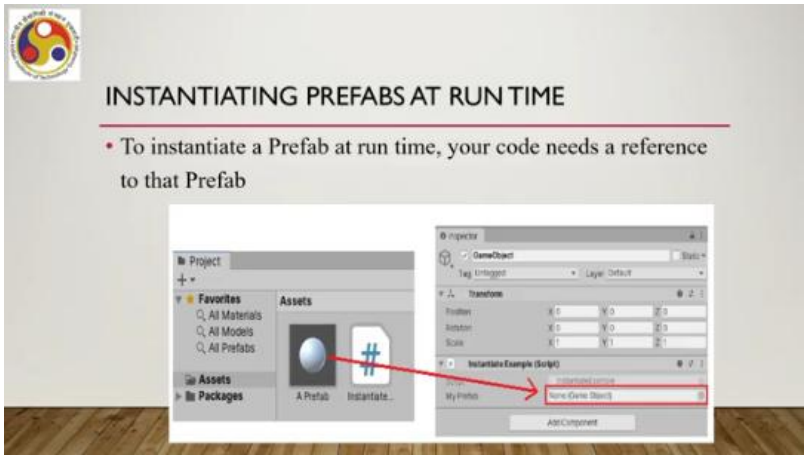
Then initialization events, such as Start and Awake; GUI events, such as OnGUI, OnMouseOver, and OnMouseDown. Physics events, such as OnCollisionEnter, OnCollisionStay, and OnCollisionExit, relate to when contact is made or broken. Similarly, other physics-related events can be OnTriggerEnter, OnTriggerStay, and OnTriggerExit. These are called when a collider is configured as a trigger. So, all these things can be utilized.



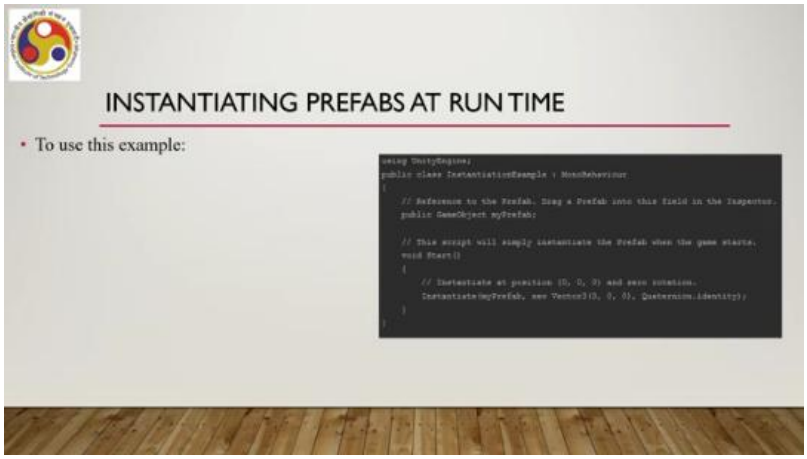
These are already there. You can use it to impose realistic behavior, say physical behavior in the physics events based functions. Similarly, interactive behavior when you are calling in GUI events functions and so on. Now, Unity passes control to an event function when events happen.

Again, most of you might be familiar with this type of thing. When there is an event, the control goes to the event handler or the event function, and then the function body is executed. Once the event function has finished executing, the control is passed back to

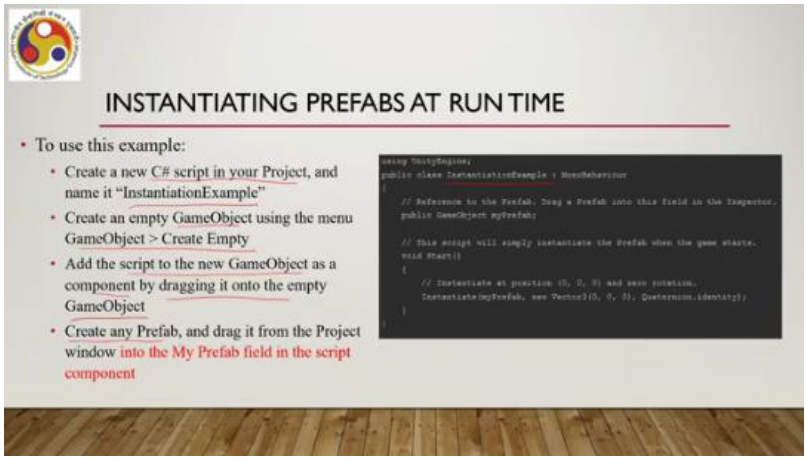
Unity, just like any other programming environment. An important concept while you are trying to work with scripts is instantiating prefabs at runtime.



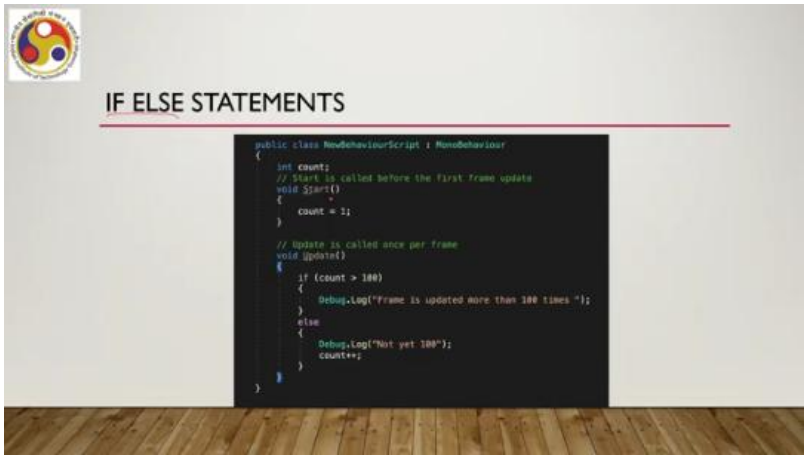
To instantiate a prefab at runtime, the code or the script that you are writing needs a reference to that prefab. The idea is shown here. In the assets class, we have a prefab, and then there is this reference. Now, in order to do that, what we need to do at the code level? Create a new C-sub script in your project and



Name it instantiation example, maybe sample code as seen here in this console; instantiation example is a new script. Create an empty game object using the menu: Game Object, Create Empty. Add the script to the new game object as a component by dragging it onto the empty game object. These operations are done with the interface. Then create any prefab and drag it from the project window into the My Prefab field in the script component.

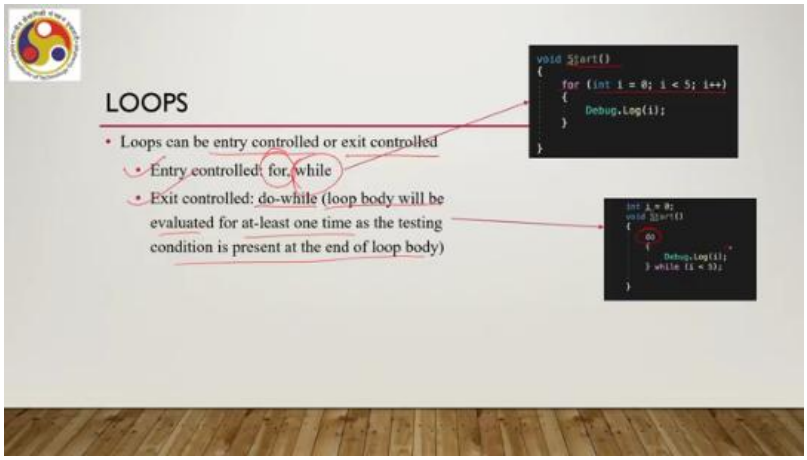


These are the options provided by the editor interface, and you can follow these steps to basically attach or instantiate prefabs at runtime. Like any programming environment, the scripting environment also supports conditional or if-else statements, such as an example shown here. Say, this Update function; within this, we can use this if count greater than 100, Debug.Log. So, this is the condition; otherwise, or else, print something else and update some variable.

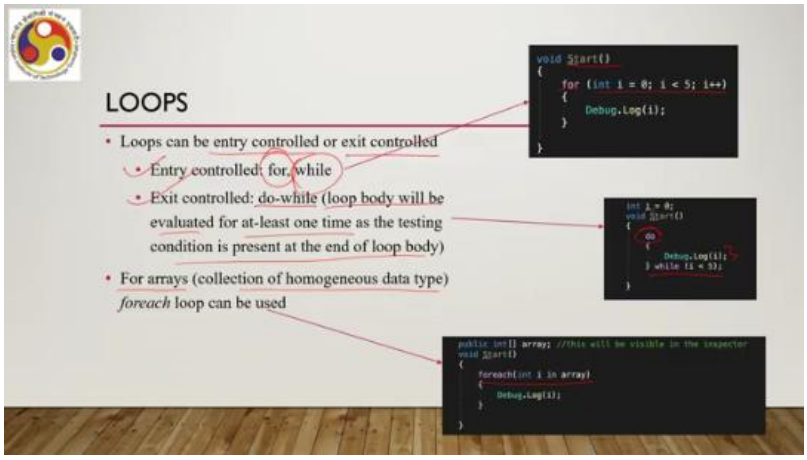


So, these are standard programming constructs, conditional statements, which are supported by scripts. Then there is loop support also, again similar to other programming environments. Now, loops can be entry-controlled or exit-controlled. When we are using constructs such as for or while, these are entry-controlled loops, as shown here. For example, in this function Start, we have entry-controlled; that is, at this stage only, loop conditions are set, and then the loop body executes.

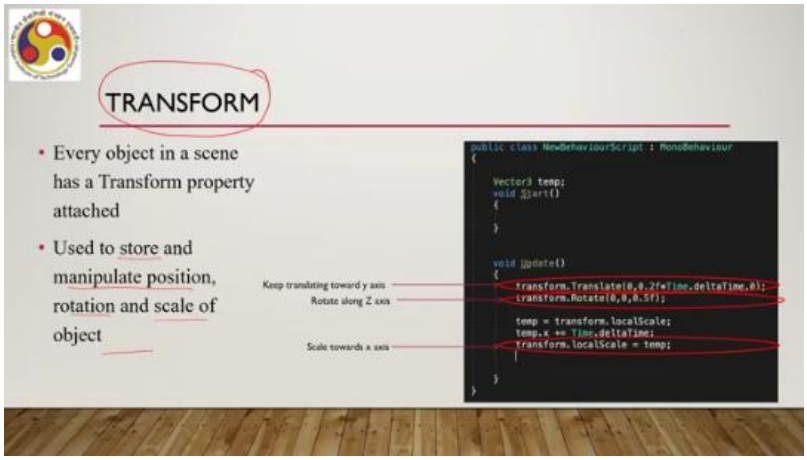
As opposed to entry controlled, we also have exit controlled. When you use the construct do while, that is an exit controlled loop; here the loop body will be evaluated at least one time as the testing condition is present at the end of the loop body, and that is why it is called exit controlled. An example is shown here. Here we have do, then this body executes, and then the condition for continuation in the loop is checked.



In the case of for, the condition is checked at the beginning, and then the body executes. In the case of do, the condition is checked after the body executes. For arrays or a collection of homogeneous data types, a for each loop can be used. That makes life easier for a programmer; instead of dealing with individual data types, we can directly use the foreach. There is a property called the transform property attached to every game object in a scene.



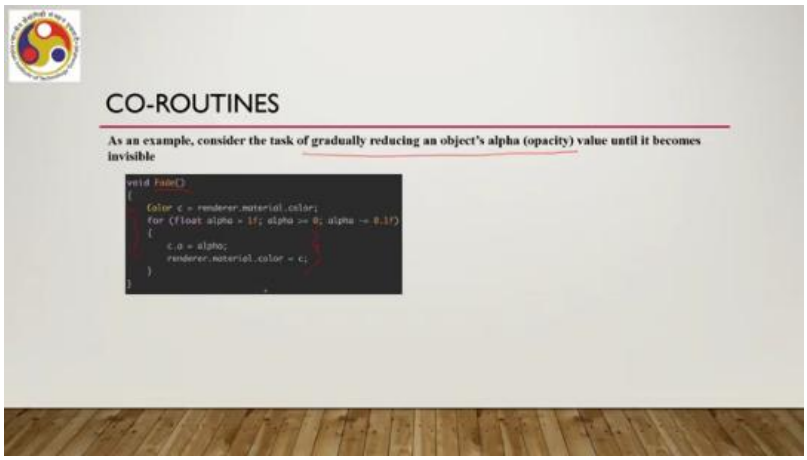
This property is used to store and manipulate the position, rotation, and scale of an object. We can actually modify this property programmatically by writing a script, using which we can update the property. For example, here under the update function, we can have transform.translate, then parameter values for where to translate, then rotation for how to rotate, then local scale for how to scale it, and so on.



So, this property we can manipulate through script to make the object behave. as per our wish, as per our interest. There is also the concept of coroutines. A coroutine allows a developer to spread tasks across several frames. This is nothing but a method that can pause execution and return control to Unity, but then continue where it left off on the following frame.

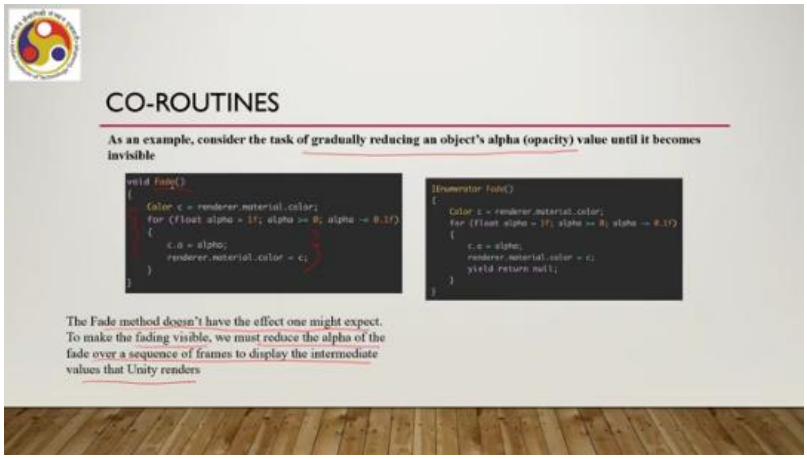
It is important to note that coroutines are not threads. They are not designed to behave like threads. Synchronous operations that run within a coroutine still execute on the main thread. So, there are no individual threads spawned out of the execution of the coroutines. For example, consider the task of gradually reducing an object's opacity value until it becomes invisible.

So, here what can be done is, in the script, we can have a function like this, fade, where the color is original color, then for certain condition, entry controlled condition, we are gradually fading. the alpha or opacity value so that reducing the value so that it eventually fades. Now, the fade method does not have the effect one might expect. To make the fading visible, we must reduce the alpha of the fade over a sequence of frames to display the intermediate values that Unity renders.

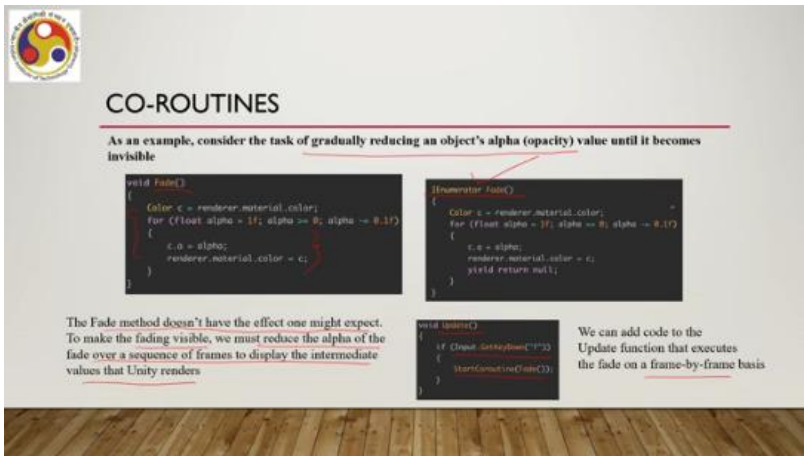


So here, now this earlier function, what happened is that we are calling the fade function, but that function may not be perceivable; the effect of that function may not be perceivable

because it is called only once. Instead, if we can use it as a coroutine, say IEnumerator fade, then across frames the fading takes place. So, then over a period of time we can perceive the change in the opacity value and then correspondingly how the object is disappearing gradually over a period of time. So, this is possible only when we implement the idea as coroutines rather than a single call function.



Now, in order to do that, we first need to define the function in this way, then we can have this update function as this coroutine, start coroutine mode, so that the fading is done on a frame-by-frame basis. So, you can see the difference in how we In order to have the effect, we modified the original implementation of the fade function shown here, this is the original one to these two functions which are modified from the original ones.



So, unless we implement it in this way, then this original fade function will be called only once, maybe for the subsequent frame, but then that effect will not be perceivable. Whereas, if we do it over a consecutive set of frames, then it will be perceivable, and the only way to do that is through the use of the concept of coroutines. There is another important function called InvokeRepeating. It invokes a method at a specific time, then repeatedly, every specified second, it calls it.

For example, here this launch projectile function, this one, when we are using the invoke repeating construct to call this launch projectile function. So, there are two parameters sent.

Of course, three parameters: one is the function name, and the other two are times in seconds. So, the first parameter says that the function starts two seconds after it is called.

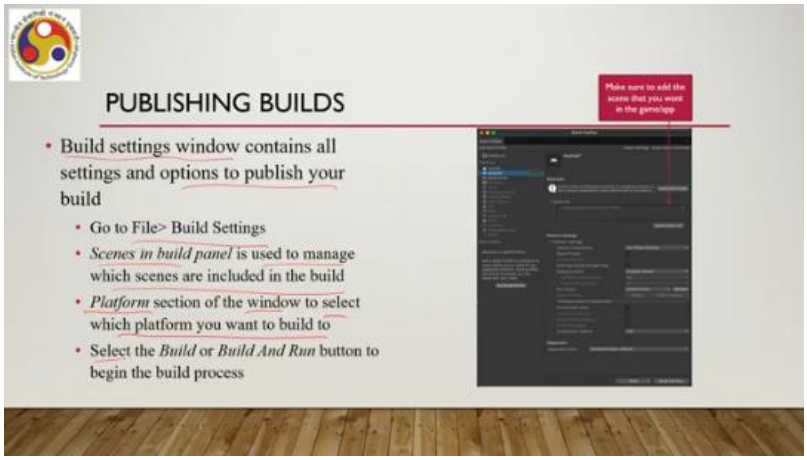


And then the second parameter, 0.3f, says that a projectile will be launched every 0.3 seconds. The projectile launching operation is done here in the launch projectile. So, every 0.3 seconds the launch projectile will be called, which means the projectile will be launched. That is the purpose of invoke repeating.

So, as you can see, these are good constructs which can be utilized for controlling the way objects behave in an immersive environment. There are many other features, of course; we cannot go through all of them in the limited scope of this lecture. So, for example, scripting supports user interaction with input devices, which, of course, is kind of obvious because that is one of the key aspects of any immersive environment: the user should be able to interact.

And that support is done through support for dealing with various inputs, such as mouse, keyboard, and buttons. And for other inputs, device-specific inputs, you may need to import SDKs where such input support may be provided and which will extend the inbuilt support of Unity. After you write the script and build the system, the last stage is to publish or to deploy the system. Now, the editor interface contains build settings, and if you open that window, it contains all settings and options to publish your build. You can go to the File Build Settings.

Then, the Scenes in Build panel is used to manage which scenes are included in the build. The platform section of the window is there to select which platform you want to build to, that means which platform you want to deploy your system. You can select the Build or Build and Run button to begin the build process. So, not all of these steps are shown here, but this is the first step that is shown. So, we have these build properties.

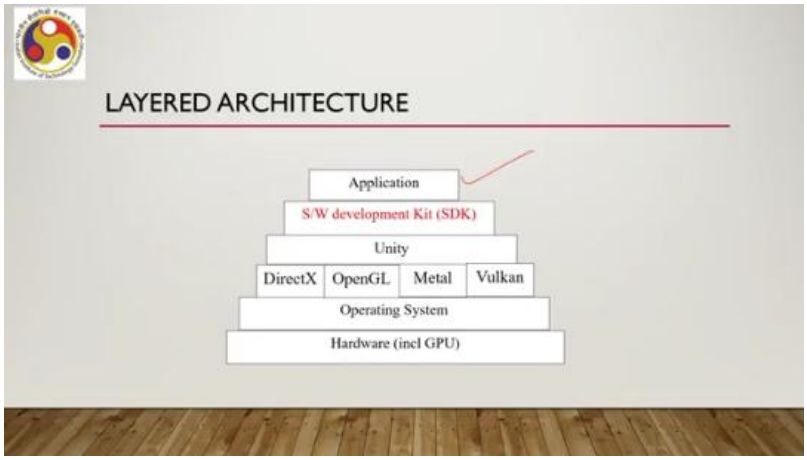


Under Builds Profiles. Then, the platform you can select. There are platform settings options. So, all these things you can do using the editor interface. So, not only are you allowed to modify the behavior of the objects in a scene, you can also use the

Use the built-in editor interface to compile and deploy your system on the specified platform as per your wish. So, that is in brief an idea of how to utilize the coding support provided in the Unity development environment to create an experience of your design of your choice. Now, so far, whatever we have discussed primarily relied on built-in support. Now, sometimes we may like to deploy our system on specified platforms, specified HMDs, or specified smart glasses.

Now, each of these specified hardware platforms or software platforms may come with their own set of features or set of functionalities that may not be part of the built-in support provided by the Unity environment. So, in that case, what we need to do is basically we need to import SDKs or packages which allow a developer to access or utilize the specific functions and functionalities provided by those specified platforms. This can be better understood in terms of the layered architecture that we have seen earlier.

So, at the top layer, there is the application. Then we talked about the Unity layer, and earlier we said that there is this SDK layer on top. So far, we have assumed this layer is nothing but a part of the Unity package which gets installed, and we are using those APIs. Below Unity, we have already said that Unity takes support of libraries such as DirectX, OpenGL, Metal, or Vulkan. Below that, we have the operating system and then hardware, including the GPU support.



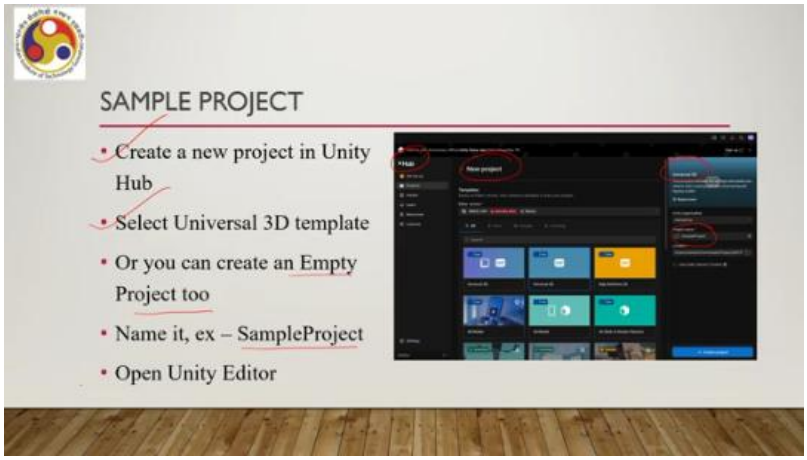
So, earlier we talked about the support provided by Unity. Let us see what we need to do for other software development kits. So, let us try to understand how SDKs work for native applications, and we will use a specific SDK called Vuforia for AR for this. Now, Vuforia is an SDK which is primarily meant to help developers deal with object recognition or computer vision issues, such as object detection, recognition, tracking, and so on. The inbuilt Unity support for these operations is not extensive, not very good, and in order to do it, seamlessly and efficiently, we require external packages. So, Vuforia is one such package. And let us try to understand how SDKs can be integrated into the Unity development environment so that we can make use of the advantages of both things, whatever the Unity environment supports as well as whatever the SDKs support. So, how can we integrate the SDK with Unity?

What are the basic steps? Install the Unity Hub first, then import the SDK package into the Unity environment, then use the functions of both whatever Unity supports as well as whatever the SDK supports. You can then compile, build, and finally deploy. So, the steps are simple and intuitive. You have to install the Hub package first, then install the SDK meant for Unity.

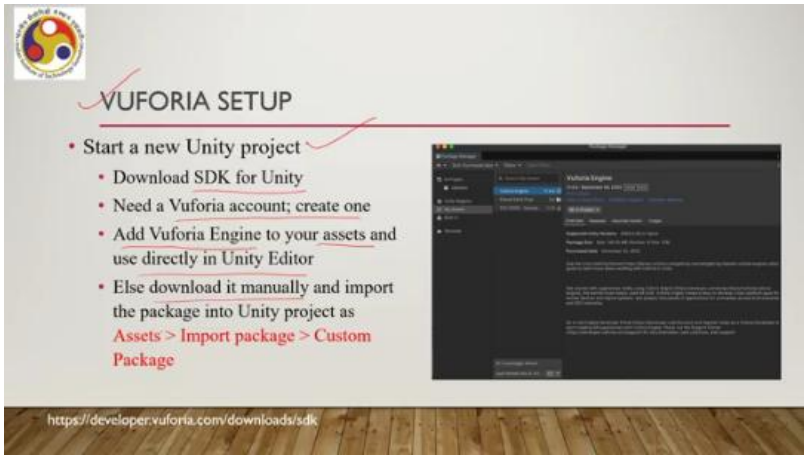
specified SDK meant for Unity, then you can use the functions to compile, build, and deploy. So, what is Vuforia? It is an SDK specifically developed for AR. It supports advanced computer vision functionality to recognize images and objects and interact with spaces in the real world. It is a cross-platform SDK, supporting multiple platforms.

such as Unity, Android, iOS, and of course, we need some license to use it; a developer's license is available. Now, let us try to understand the concept with respect to a sample project. So, first we need to create a new project in the Unity Hub, as shown here: new project, Hub interface. There we need to select the universal 3D template, as shown here. Alternatively, you can create an empty project also, as per your choice, and give a name to the project, such as sample project, as shown here: project name.

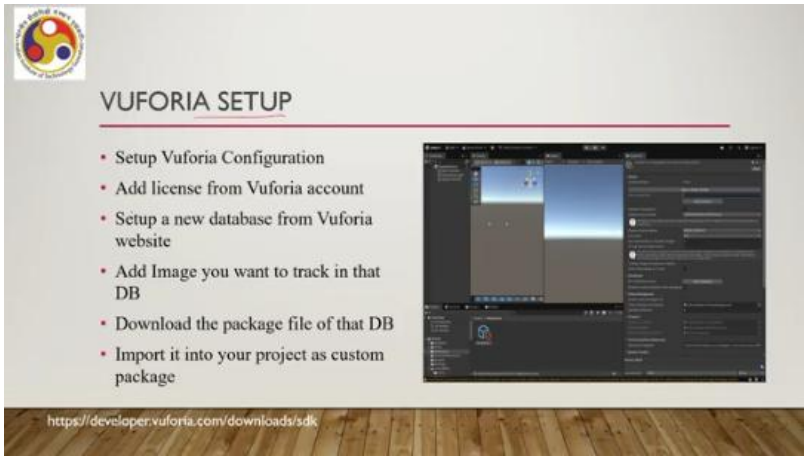
and then open the editor interface. Then you need to have the Vuforia SDK setup done. So, you can start a new Unity project, then in that project you can download the SDK for Unity, Vuforia SDK for Unity. In order to do that, you need a Vuforia account; you need to create one and then arrange for a license. Then you can add the Vuforia engine.



to your assets and use it directly in the Unity editor. So, once the engine is added to the assets, you can start using it directly without any further installation. Otherwise, you can download it manually and import the package into the Unity project as assets, import packages, and then custom package. So, some of the steps are shown here. So, you are downloading it, My Assets, Vuforia engine, the engine version is shown, and then other things are taken care of.



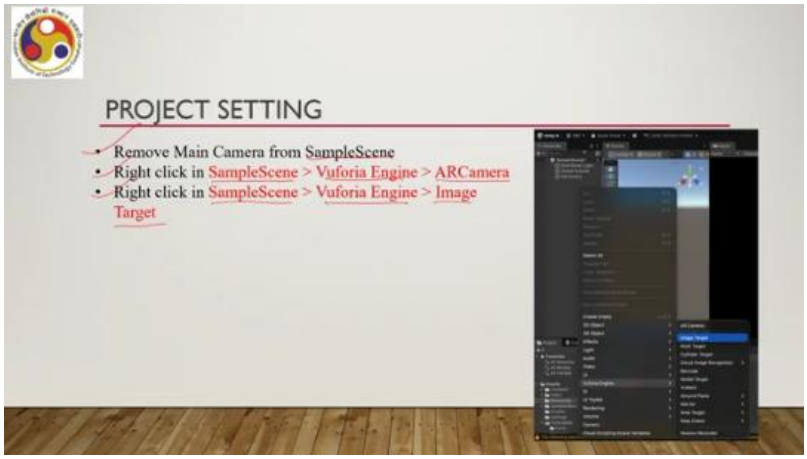
To know more details of how to use Vuforia, how to install it, you can go to developer.vuforia.com, and there you can get to know more details. Also, you can download the corresponding SDK for Unity. Now, under Vuforia setup, you need to set up the Vuforia configuration. You need to add a license from your Vuforia account.



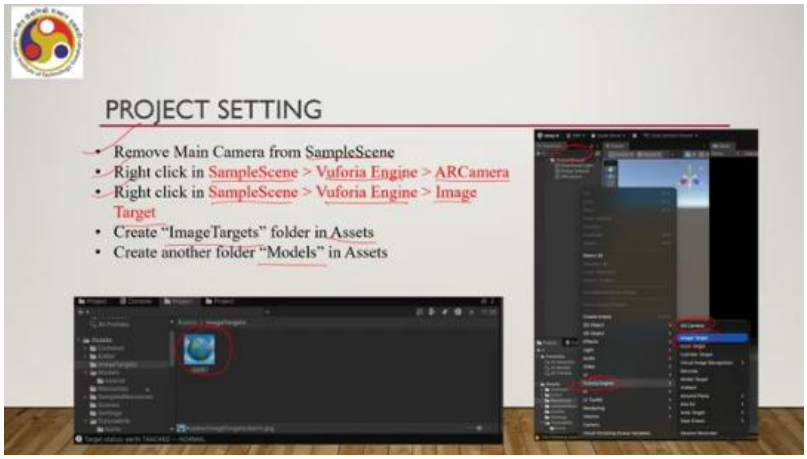
Set up a new database from the Vuforia website. This database you will require for the recognition task. Add images you want to track in that database. So, if you are building a project where you want to detect some images, recognize some images, say, for example, a marker-based AR system where you need to recognize the markers, you need to take a picture, multiple pictures, and add them to the database you have set up so that the Vuforia engine can recognize the image. Download the

Package file of that database and import it into your project as a custom package. Again, some of the steps are shown here. You can see the sample scene, then these other options are shown on the editor interface. Under assets, you can see Vuforia-related assets; the add database option is here, and so on. Now, once those are done, you can go to the project settings where you can remove the

main camera from the sample scene, select the AR camera by following the sequence: sample scene, Vuforia engine, AR camera, then select the image target by following the steps: sample scene, followed by Vuforia engine, followed by image target. As shown here: sample scene, Vuforia engine, AR camera, image target; these options are available here. Then you need to create an image targets folder in assets, and create another folder, models, in assets.

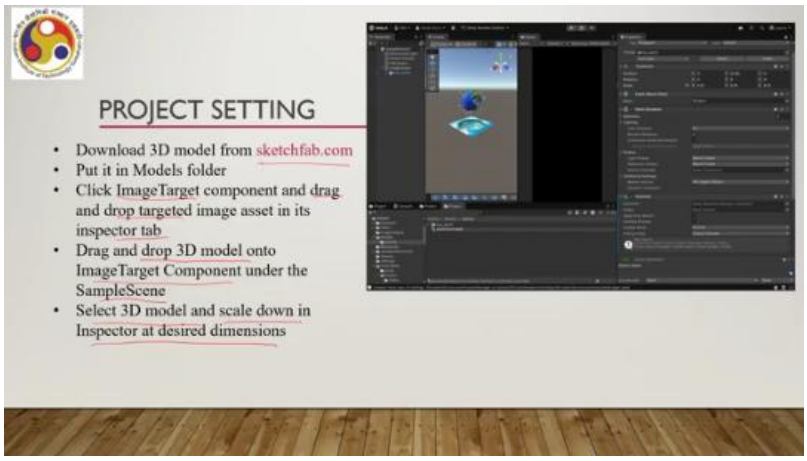


So, this is the target image. So, you have to create it under the assets image targets folder. Put the target image in the image targets folder as shown here. So, this is the target image. The sample project will be trying to recognize this for a marker-based AR application.

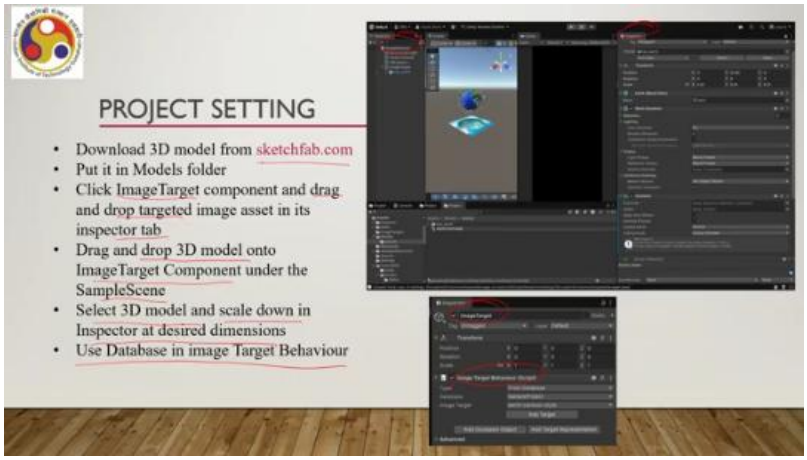


So, this image is captured and put in the asset folder so that the Vuforia engine can recognize it. Then we need to download a 3D model from sketchfab.com. So, that is for the sample project that has been utilized. Put it in the models folder, click or select the image target component, and drag and drop the targeted image asset in its inspector tab. Then drag and drop the 3D model onto the image target component under the sample scene. Select the 3D model and scale it down in the inspector to the desired dimensions.

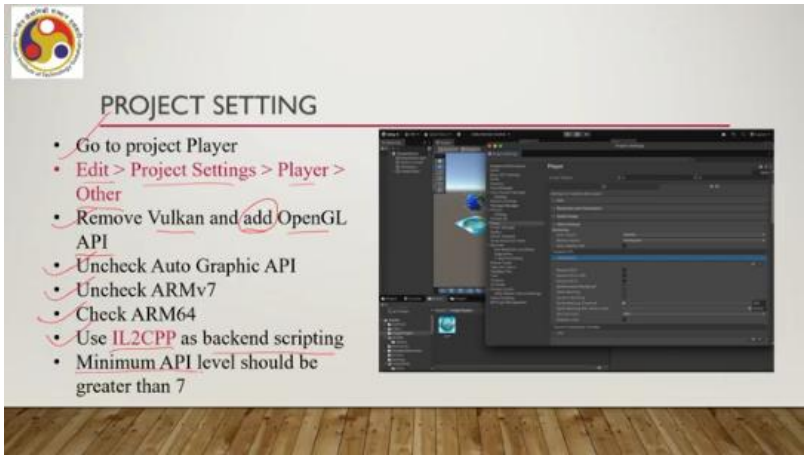
Again, some of these steps are shown here. So, the sample scene you can see, the inspector interface you can see. Then we have various options under the inspector interface. As mentioned here, those settings can be done using the inspector interface. So, we need to use the database in the image target behavior.



So, here we have the image target, the image target behavior script. So, that is how we can attach or modify the behavior of the target by writing a script. Then we need to go to the player following this sequence: Edit, Project Settings, Player, Other. We need to remove Vulkan and add the OpenGL API.

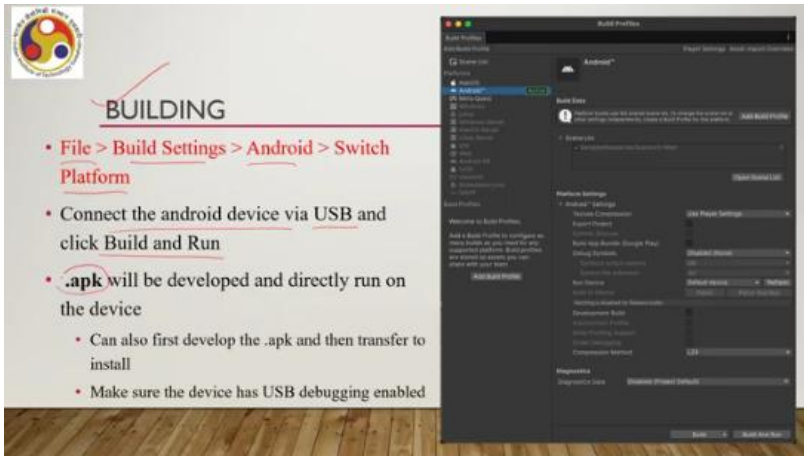


We need to uncheck AutoGraphics API, uncheck ARM version 7, check ARM 64, and use IL2CPP as the backend scripting. We need to ensure that the minimum API level is greater than 7. So, these options we can set using this player interface. So, some of the settings are shown here; you can see that whatever is mentioned in these steps is followed on the interface. Now, these settings, of course, are application-specific.



I am specifically mentioning these settings for this sample project. For other projects, of course, the same settings will not work. You have to consider your own settings. You have to set your own configurations to make your project work. Once those settings are done, we go to the building stage.

We follow the sequence: File, Build Settings, Android, Switch Platform, then connect the Android device. So, we are assuming that it is implemented on an Android platform. So, you connect the Android device via USB and select the Build and Run options. An APK file or app file will be created and directly deployed on the device. However, you can do it separately.



You can first develop the APK and then transfer it to install on the device. And you have to make sure that the device has USB debugging enabled. Again, some of the steps are shown here. On the Android platform, we are using the build options. For platform settings, we are using the settings.

Other settings are as per requirement. Again, it may change from one platform to another. So with all those settings done and all those scripts to change the behavior, what do we get? So in the sample project, we can develop one marker-based AR. So this Earth image is the marker.



Suppose this marker is shown on this display. And if we capture it, it can be on paper also. A colored image has to be there on paper also. For Vuforia to recognize, this is the target image. And once that is captured by your device, which is running the APK file, then a 3D model as shown here is displayed in place of the marker.

So, with a 3D model, you can get to see. So, the marker can be placed on a mobile screen, on a desk; a paper-based marker can be there. Anything is okay as long as it matches with the original image, color, shape, and size. And once that is done, then the Vuforia engine takes care of recognizing the marker, and then the APK file, the script, is there to perform operations once the marker is recognized.

So, that is how you can install an SDK to add to the functionalities of the inbuilt environment provided by the Unity development environment. So, for example, here by utilizing the Vuforia engine, you can add to the image recognition functionalities of the built-in environment. Similarly, for various devices to access device-specific data, sensory data, or input data, you can utilize device-specific SDKs in a similar way as shown in this discussion. So, let us summarize what we have learned so far. What we covered: how to build and deploy systems, how to use scripting for developing a customized experience, and how to incorporate SDKs for device-specific support.

So, all these are aimed toward giving you some idea of how to relate theory to practice. So, building and deployment These are programming tasks, primarily. Some require less programming; some require more programming. If you want to have a customized experience, certainly you have to go for scripting, which involves more programming.

If you want a basic system with basic functionalities, you do not need to go for scripting. The IDE itself provides much menu-based support for building such a system. And if you want more functionalities than the support provided by the Unity development environment, you need to import SDKs. So, these are basic concepts, but overall the idea is that whatever theory we have learned, we can convert that theory into features or functionalities supported by the development environment itself or in conjunction with some other SDKs; these functionalities can be supported. So, with that, I would like to conclude this part of the lecture. Our next part will be the concluding remarks, following which we will conclude the overall course. So, I hope you could follow the concepts. Of course, many details are omitted.

The discussion was kept at a very simple level just to give a basic understanding. For more details on how to build and deploy systems, how to write code, you can refer to the Programmer's Manual and Developer's Manual that you can find on the respective websites. So, thank you. Looking forward to meeting you in the next part of the lecture.

Goodbye and happy learning.